



UNIVERSITA' DEGLI STUDI DI
NAPOLI FEDERICO II

Scuola Politecnica e delle Scienze di Base
Corso di Laurea in Ingegneria Informatica

Corso di **Network and Cloud Infrastructures**

**Progetto finale: Analisi e Mitigazione di Attacchi DoS in una
Rete Simulata**

Anno Accademico 2025/2026

Link Github:

https://github.com/giovannistanco/NCIS_unina/tree/main/2026/cleopatra_dabrosca

Candidati

Gianni Cleopatra DE9000169

Valerio D'Abrosca DE9000006

Introduzione

Nelle architetture Software-Defined Networking, la centralizzazione logica del Control Plane offre numerosi vantaggi, specialmente nell'ambito della programmabilità e della scalabilità, ma al costo di una maggiore latenza e overhead di elaborazione durante la gestione dei nuovi flussi.

L'obiettivo di questo progetto è analizzare vantaggi e svantaggi di queste architetture mediante una rete simulata Mininet gestita dal controller Ryu con protocollo OpenFlow 1.3. L'analisi si focalizza sugli effetti di numerose tipologie di attacchi Denial of Service (DoS) valutati prima su un controller base dotato unicamente di un meccanismo di L2 Forwarding (codice fornito come parte del materiale didattico del corso), e successivamente su una versione avanzata appositamente sviluppata per l'individuazione e mitigazione di queste minacce.

In particolare, sono stati analizzati gli effetti degli attacchi PACKET-IN Flood, Spoofed IP/MAC Flood, ARP Poisoning e TCAM Flow Table Exhaustion, mentre sono stati progettati e introdotti meccanismi di rilevazione e mitigazione contro gli attacchi UDP Flood a banda costante e impulsivo, TCP SYN Flood e ICMP Flood.

Oltre al rafforzamento della sicurezza della rete, sono state introdotte numerose migliorie al controller OpenFlow: analisi granulare del traffico mediante statistiche di flusso, gestione dinamica delle soglie di allarme mediante media mobile delle metriche del traffico, introduzione di un'interfaccia REST API per la gestione esterna delle regole da parte dell'amministratore di rete e divisione del controller in microservizi disaccoppiati con comunicazione mediante la coda degli eventi di Ryu.

Architettura del Sistema

Sono state usate due topologie di rete Mininet: la prima, definita nel file "topology.py" e usata per i test di individuazione e mitigazione, è presente nell'immagine a seguire, mentre la seconda, definita nel file "control_plane_testbed.py" e usata per gli stress test del controller, è una versione semplificata della prima costituita da un solo switch connesso a tutti gli host.

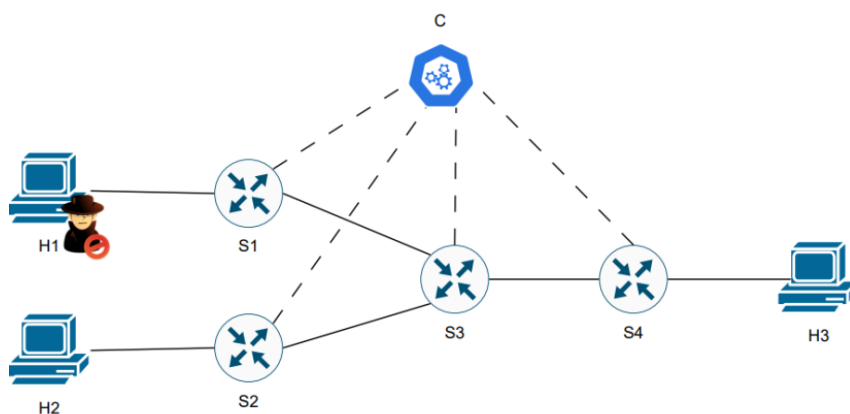


Figura 1 - Topologia rete Mininet (Fonte: materiale didattico)

Il controller realizzato per questo progetto, il cui orchestratore è definito nel file “modular_controller.py”, è diviso in cinque microservizi:

- **L2 Forwarding**, definito nel file “l2_forwarding.py”, gestisce il normale istradamento del traffico e l’apprendimento degli indirizzi MAC;
- **Monitor**, definito nel file “monitor.py”, raccoglie ogni secondo le statistiche di flusso dagli switch mediante richieste FlowStats e invia report alla Detection;
- **Detection**, definito nel file “detection.py”, basandosi sui report ricevuti dal Monitor calcola la media mobile di byte_count e packets_count per individuare anomalie nel traffico e, in caso affermativo, determina che tipo di attacco è in corso basandosi su euristiche. Determinato l’attaccante e il tipo di attacco richiede l’installazione di regole di DROP mirate da parte della Mitigation;
- **Mitigation**, definito nel file “mitigation.py”, tiene traccia e installa regole di DROP per bloccare flussi malevoli;
- **REST API**, definito nel file “modular_controller.py”, permette ad un amministratore di rete di ottenere la topologia e visualizzare, aggiungere o rimuovere regole di DROP mediante richieste HTTP.

Ambiente di Test

Il setup dell’ambiente di test è avvenuto seguendo le specifiche presenti nel materiale didattico, riportate nelle slide relative a Mininet. Di seguito è riportata la slide in questione:

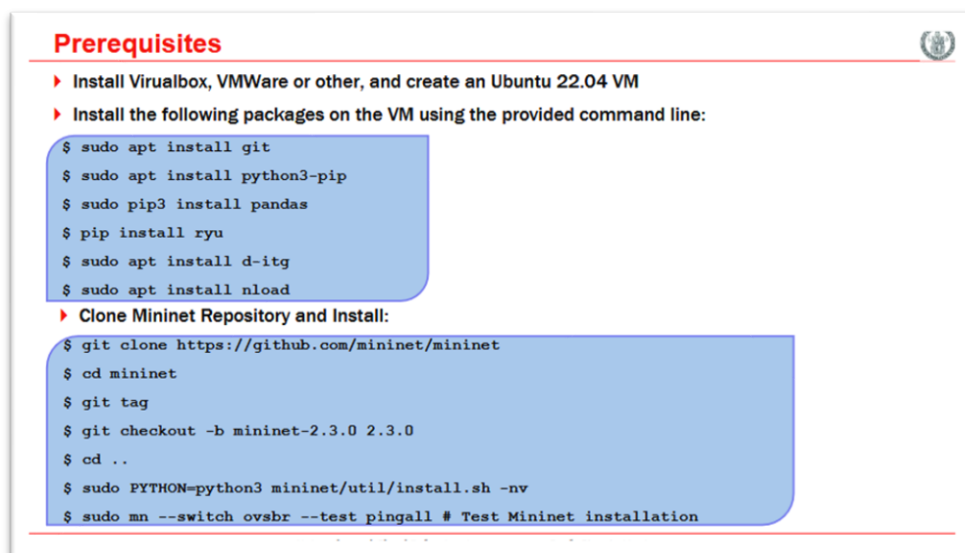


Figura 2 - Prerequisiti ambiente di test (Fonte: materiale didattico)

A questi requisiti sono stati aggiunti hping3 e scapy, installabili con i seguenti comandi

```
> sudo apt install hping3
> pip install scapy
```

Guida al Testing

Struttura del Codice

Il codice sorgente è stato organizzato in due moduli principali così da separare le testbed sperimentali dal controller Ryu:

```
/
├── attacchi/           # script python per la simulazione degli attacchi DoS
├── controller/         # script python costituenti i microservizi del controller Ryu
└── simple_controller.py # controller Ryu con L2 Forwarding
```

- La cartella “attacchi/” contiene gli script python per lanciare gli attacchi DoS, divisi tra “control_plane_testbed.py” contenente gli attacchi diretti al Control Plane e “attack_*.py” per gli attacchi diretti al Data Plane.
- La cartella “controller/” contiene gli script python costituenti i microservizi e l’orchestratore del controller Ryu.
- Lo script “simple_controller.py”, fornito con il materiale didattico del corso, è un semplice controller Ryu contenente solo la componente di L2 Forwarding.

Scenari di Test

Per effettuare un test è necessario avviare due terminali:

- nel primo si esegue il controller Ryu appropriato, scelto tra il “simple_controller.py” per i test in cui si vuole verificare l’effetto dannoso per la rete dell’attacco DoS e il “controller/modular_controller.py” per i test in cui si vuole verificare l’individuazione e conseguente mitigazione dell’attacco;
- nel secondo si esegue lo script python contenente l’attacco o gli attacchi appropriati.

Gli scenari di attacco si dividono tra:

- attacchi al solo Control Plane, di cui l’analisi si è limitata all’effetto sulla rete. Viene lanciato lo script “control_plane_testbed.py” mentre è attivo il controller Ryu “simple_controller.py” e si osservano i log nel terminale e nel file generato. Mediante la command line di mininet, aperta direttamente dallo script del testbed, è possibile lanciare attacchi e stress test mediante i comandi **attack** (solo attacco), **run** (attacco con una sola istanza) e **stress** (attacco con n istanze), fermare e pulire la rete mediante i comandi **stop** e **reset**, osservare i flussi attivi mediante **flows** e **watchflows**, pulire i flussi mediante **freshflow** e controllare la tabella ARP di un host mediante **arp**;

```
# TERMINALE CONTROLLER
> ryu-manager simple_controller.py

# TERMINALE ATTACCO
> cd attacchi/
> python3 control_plane_testbed.py
mininet> stress 1 5 10 output.csv
```

- attacchi al Data Plane e Control Plane, per i quali sono stati effettuati test sia con che senza difesa. Per testare uno di questi scenari si sceglie prima un attacco tra i quattro selezionati per il progetto (UDP statico e impulsivo per gli attacchi al solo Data Plane, TCP SYN Flood e ICMP Flood per gli attacchi a Data Plane e Control Plane). Scelto l'attacco viene lanciato lo script corrispondente dapprima mentre è attivo il controller Ryu "simple_controller.py" e si osservano i log da terminale per verificarne gli effetti dannosi sulla rete, per poi pulire la rete e ripetere l'esperimento con il medesimo script ma attivando il controller Ryu "modular_controller.py", osservando poi i log da terminale per verificare l'installazione delle regole di DROP (terminale con il controller) e la mancanza degli effetti sulla rete riscontrati nell'esperimento precedente (terminale con l'attacco).

```
TEST SENZA DIFESA

# TERMINALE CONTROLLER
> ryu-manager simple_controller.py

# TERMINALE ATTACCO
> cd attacchi/
> python3 attack_*.py

# sudo mn -c      prima di passare al test successivo

TEST CON DIFESA

# TERMINALE CONTROLLER
> cd controller/
> ryu-manager modular_controller.py

# TERMINALE ATTACCO
> cd attacchi/
> python3 attack_*.py
```

Admin REST API

È inoltre possibile interagire con la REST API (classe AdminRestController nel file "controller/modular_controller.py") del controller mediante richieste http.

Esempi di richieste:

```
# Topologia della rete in formato json
> curl -X GET "http://localhost:8080/api/v1/topology" -H "Accept: application/json"

# Aggiunta di una regola di DROP ad ogni switch per il MAC 00:00:00:00:00:01 con hard_timeout = 60
> curl -X POST "http://localhost:8080/api/v1/policy/block" -H "Content-Type: application/json" -d
'{"mac": "00:00:00:00:00:01", "duration": 60}'

# Elenco di ogni regola di DROP associata al MAC 00:00:00:00:00:01 per ogni switch
> curl -X GET "http://localhost:8080/api/v1/policy/block?mac=00:00:00:00:00:01" -H "Accept:
application/json"

# Rimozione di ogni regola di DROP associata al MAC 00:00:00:00:00:01 per ogni switch
> curl -X DELETE "http://localhost:8080/api/v1/policy/block/00:00:00:00:00:01" -H "Accept:
application/json"
```